Chapter 1

The YOLO Training Pipeline

1. What "Training" Actually means

At its core an "untrained" neural network is a mathematical function filled with random weights.

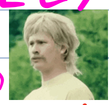
Training is the algorithmic process of systematically adjusting these weights until the network's outputs match our desired ground-truth labels.

2. Forward Propagation ("guessing phase")

- An image (usually a batch of images) is passed into the network.
- Pixel values undergo millions of mathematical operations from the backbone to the decision head.
- Final output is the network prediction: (box coordinates (x, y, w, h), Objectness Score [0,1] % of object ?, Class Probabilities [0,1] for each class)

3. Loss Function

SCYLLA IoU



One of the famous ones used for YOLO is SIoU, combines:

Angle cost: Angle-aware, reduces the issue of "wandering" cause of high DoF.

Distance cost: Make predicted boxes fall near ground truths, takes angle cost into account.

Shape cost: Aspect ratio missmatch.

math here is a bit out of scope

IoU cost: Normal intersection over union subtracted by 1.

↳ used for Imbalanced data as well. ★★★

Alongside Focal Loss for object detection itself, it improves upon Cross Entropy Loss by reducing the impact of the majority class samples

$$CE(P_t) = -\log(P_t)$$

$$FL(P_t) = -(1-P_t)^{\gamma}\log(P_t)$$

usually 2 as a starter
the larger it gets, the more we down-weight the majority class (easy examples)

* $P_t \approx 1$ highly confident: doesn't affect the loss much since its an easy example
  $L \approx 0$ since $(1-P_t)^{\gamma} \approx 0$
* $P_t \approx 0$/low uncertain/poor predictions: highly affects the loss & Keeps it intact

4. Backpropagation VS. Gradient Descent

⊛ Backprop.: the calculus Chain Rule, it's the algorithm that calculates the gradients: → in PyTorch `loss.backward()`

$$\frac{\partial L}{\partial w_{\theta_t}} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z} * \frac{\partial z}{\partial w}$$

Does NOT change any weights

⊛ Gradient Descent (Optimizer) in PyTorch → `optimizer.step()`

An Optimization algorithm that takes the gradients from backprop. then it adjusts the parameters to minimize the loss

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t)$$

→ graidents of the Loss function w.r.t current parameters θ_t

learning rate (alpha)

⊛⊛ Adam ⊛⊛

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$\hat{m}_t$: First moment } Momentum principles
$\hat{v}_t$: Second moment }
ϵ: Smoothing term to prevent $\frac{\dots}{0}$

① Compute $\nabla L(\theta_t)$

② Update moments

$$\hat{m}_t = \beta_1 m_{t-1} + (1-\beta_1) * \nabla L(\theta_t)$$

$$\hat{v}_t = \beta_2 v_{t-1} + (1-\beta_2) * \nabla L(\theta_t)^2$$

③ Bias Correction

$$\hat{m}_t = \frac{\hat{m}_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{\hat{v}_t}{1-\beta_2^t}$$

β_1 & β_2: exponential decay rates
typically $\beta_1 = 0.9$
$\beta_2 = 0.999$

⊛ Adam uses L2 (ridge) regularization in the loss function ⊛

⊛⊛ AdamW ⊛⊛ W: Weight decay

$$\theta_t = \theta_{t-1} - \alpha \lambda \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

λ: weight decay

→ Subtracted directly from the prameter rather than in the loss function

- Why it is better??

① Proper Regularization: In Adam, large historical gradrents face less weight decay than weights with small gradients But AdamW ensures all weights decay at a rate proportional to their size

→ λ is no longer inside ∇L so if ∇L is large it won't effect

② Hyperparameter Independence: In Adam, changing (α) affects weight decay
But AdamW we can tune α & λ independently from each other.

⊛ لازم نغير (α ، λ) مو بعض لانهم متغيرين. ⊛

✪ Extra ✪ Regularization: L1 (Lasso), L2 (Ridge), ElasticNet (combination of L1&L2), Dropout, Early Stopping

5. One Batch VS. One Epoch

✪ Batch: Divide the dataset into smaller chuncks called Batches (64 images)

- One complete cycle of Forward prod → loss → Backprop → weight update for every batch this is called iteration/step

✪ Epoch: An epoch is completed when the model sees the dataset exactly once

e.g. 1600 image dataset & batch size of 16 → 100 steps to complet one epoch

6. Validation & Checkpoints

After an epoch is finished, the model is tested on the validation set only performing forward prop.

If the validation metrics improve → Save a copy of the model's weights

This is called a checkpoint best.pt